PROGRAM 1 1.c Matrix Addition

C SOURCE CODE 1.c

```c
#include<stdio.h>
int main()
  {
    int a[10][10],b[10][10],c[10][10];
    int r1,r2,c1,c2;
    int i,j;
    printf("enter no of rows and columns of first matrix");
    scanf("%d%d",&r1,&c1);
    printf("enter no of rows and columns of second matrix");
    scanf("%d%d",&r2,&c2);
    if(c2==r1)
    {
      printf("enter the elments for first matrix");
      for(i=0;i<r1;i++)
      {
        for(j=0;j<c1;j++)
        {

          scanf("%d",&a[i][j]);
        }
      }
      printf("enter the elments for second matrix");
      for(i=0;i<r2;i++)
      {
        for(j=0;j<c2;j++)
        {
          scanf("%d",&b[i][j]);
        }
     }
     for(i=0;i<r1;i++)
      {
        for(j=0;j<c2;j++)
        {
          c[i][j]=a[i][j]+b[i][j];
        }
      }
      printf("The output marix is given below \n");
      for(i=0;i<r1;i++)
      {
        printf("\n");
        for(j=0;j<c2;j++)
        {
          printf("%d",c[i][j]);
          printf("\t");
        }
      }
    }
    else
    {
       printf("error 404 not found the coulmn of first matrix and row of second matrix is  mismatche
d");
    }
return 0;
}

```

PROGRAM OUTPUT Execution Verification

```
Enter row & column sizes of 1st matrix: 2 2
Enter 4 elemnts for 1st matrix: 1 2 3 4
Enter 4 elemnts for 2nd matrix: 5 6 7 8
Added:
```

```
6 8
10 12
```

PROGRAM 2 2.c Addition of Two Polynomials Using Structures

C SOURCE CODE 2.c

```c
#include<stdio.h>
struct poly
{
int coeff;
int exp;
};
void main()
{
struct poly p1[10],p2[10],sum[10];
int n1,n2,i,j,k;
printf("enter the no of elements of p1");
scanf("%d",&n1);
printf("enter the coefficient and exponential:\n");
for(i=0;i<n1;i++)
{
scanf("%d%d",&p1[i].coeff,&p1[i].exp);
}
printf("enter the no of elements of p2");
scanf("%d",&n2);
printf("enter the coefficient and exponential:\n");
for(j=0;j<n2;j++)
scanf("%d%d",&p2[j].coeff,&p2[j].exp);
i=j=k=0;
while(i<n1&&j<n2)
{
if(p1[i].exp==p2[j].exp)
{
sum[k].coeff=p1[i].coeff+p2[j].coeff;
sum[k].exp=p1[i].exp;
i++;
k++;
j++;
}
else if(p1[i].exp>p2[j].exp)
{
sum[k]=p1[i];
i++;
k++;
}
else
{
sum[k]=p2[j];
j++;
k++;
}
}
while(i<n1)
{
sum[k]=p1[i];
k++;
i++;
}
while(j<n2)
{
sum[k]=p2[j];
j++;
k++;
}
printf("resultant polynomial:");
for(i=0;i<k;i++)
{
printf("%dx^%d",sum[i].coeff,sum[i].exp);
if(i≠k-1)
{
printf("+");
```

```
66  }
67  }
68  }
```

PROGRAM OUTPUT Execution Verification

```
Enter number of elements for 1st Polynomial: 2
Enter number of elements for 2nd Polynomial: 2

Enter 2 elements for 1st polynomial:
4 2
3 1

Enter 2 elements for 2nd polynomial:
5 2
2 0
Sum:
(9**2) + (3**1) + (2**0)
```

PROGRAM 3 3.c Tuple Representation of Sparse Matrix

C SOURCE CODE 3.c

```c
#include<stdio.h>
int main()
{
int a[10][3],t[100][3];
int r,c,i,j,k=1;
printf("enter row and column");
scanf("%d%d",&r,&c);
printf("enter elements of matrix:\n");
for(i=0;i<r;i++)
{
for(j=0;j<c;j++)
{
scanf("%d",&a[i][j]);
}
}
t[0][0]=r;
t[0][1]=c;
t[0][2]=0;
for(i=0;i<r;i++)
{
for(j=0;j<c;j++)
{
if(a[i][j]!=0)
{
t[k][0]=i;
t[k][1]=j;
t[k][2]=a[i][j];
k++;
t[0][2]++;
}
}
}
printf("tuple representation:\n");
printf("row column value:\n");
for(i=0;i<k;i++)
printf("%d\t%d\t%d\n",t[i][0],t[i][1],t[i][2]);
return 0;
}

```

PROGRAM OUTPUT Execution Verification

```
Enter row & column size: 3 3
Enter 9 elements: 0 0 5 0 8 0 0 0 0
(3, 3, 2)
(0, 2, 5)
(1, 1, 8)
```

PROGRAM 4 4.c Sparse Matrix Addition

C SOURCE CODE 4.c

```c
/* Sparse Matrix Addition */

#include <stdio.h>

int main() {
    int rows, cols, non_zero1, non_zero2;
    printf("Enter total number of rows and columns for the matrices: ");
    scanf("%d%d", &rows, &cols);
    printf("Enter number of non-zero elements for Matrix 1: ");
    scanf("%d", &non_zero1);
    printf("Enter number of non-zero elements for Matrix 2: ");
    scanf("%d", &non_zero2);

    int sparse1[non_zero1 + 1][3];
    int sparse2[non_zero2 + 1][3];
    int sum[non_zero1 + non_zero2 + 1][3];

    sparse1[0][0] = rows; sparse1[0][1] = cols; sparse1[0][2] = non_zero1;
    sparse2[0][0] = rows; sparse2[0][1] = cols; sparse2[0][2] = non_zero2;
    sum[0][0] = rows; sum[0][1] = cols;

    printf("\nEnter Triplets for Matrix 1 (Row, Column, Value):\n");
    for (int i = 1; i ≤ non_zero1; i++) {
        scanf("%d%d%d", &sparse1[i][0], &sparse1[i][1], &sparse1[i][2]);
    }

    printf("\nEnter Triplets for Matrix 2 (Row, Column, Value):\n");
    for (int i = 1; i ≤ non_zero2; i++) {
        scanf("%d%d%d", &sparse2[i][0], &sparse2[i][1], &sparse2[i][2]);
    }

    int i = 1, j = 1, k = 1;

    while (i ≤ non_zero1 && j ≤ non_zero2) {
        if (sparse1[i][0] == sparse2[j][0]) {
            if (sparse1[i][1] == sparse2[j][1]) {
                sum[k][0] = sparse1[i][0];
                sum[k][1] = sparse1[i][1];
                sum[k][2] = sparse1[i][2] + sparse2[j][2];
                i++; j++; k++;
            }
            else if (sparse1[i][1] < sparse2[j][1]) {
                sum[k][0] = sparse1[i][0];
                sum[k][1] = sparse1[i][1];
                sum[k][2] = sparse1[i][2];
                i++; k++;
            }
            else {
                sum[k][0] = sparse2[j][0];
                sum[k][1] = sparse2[j][1];
                sum[k][2] = sparse2[j][2];
                j++; k++;
            }
        }
        else if (sparse1[i][0] < sparse2[j][0]) {
            sum[k][0] = sparse1[i][0];
            sum[k][1] = sparse1[i][1];
            sum[k][2] = sparse1[i][2];
            i++; k++;
        }
        else {
            sum[k][0] = sparse2[j][0];
            sum[k][1] = sparse2[j][1];
            sum[k][2] = sparse2[j][2];
            j++; k++;
```

```c
        }
    }

    while (i ≤ non_zero1) {
        sum[k][0] = sparse1[i][0];
        sum[k][1] = sparse1[i][1];
        sum[k][2] = sparse1[i][2];
        i++; k++;
    }

    while (j ≤ non_zero2) {
        sum[k][0] = sparse2[j][0];
        sum[k][1] = sparse2[j][1];
        sum[k][2] = sparse2[j][2];
        j++; k++;
    }

    sum[0][2] = k - 1;

    printf("\nResultant Sparse Matrix (Triplet Representation):\n");
    printf("Row\tCol\tValue\n");
    for (int m = 0; m < k; m++) {
        printf("%d\t%d\t%d\n", sum[m][0], sum[m][1], sum[m][2]);
    }

    return 0;
}
```

PROGRAM OUTPUT Execution Verification

```
Enter total number of rows and columns for the matrices: 3 3
Enter number of non-zero elements for Matrix 1: 2
Enter number of non-zero elements for Matrix 2: 2

Enter Triplets for Matrix 1 (Row, Column, Value):
0 0 5
1 1 3

Enter Triplets for Matrix 2 (Row, Column, Value):
0 0 2
2 2 4

Resultant Sparse Matrix (Triplet Representation):
Row     Col     Value
3       3       3
0       0       7
1       1       3
2       2       4
```

PROGRAM 5 5.c

Transpose of Sparse Matrix

C SOURCE CODE

5.c

```c
#include <stdio.h>
int main(){
int s[20][3],t[20][3];
int i,j,k=1;
printf("enter rows and columns and number of non zeros:");
scanf("%d %d %d",&s[0][0],&s[0][1],&s[0][2]);
printf("enter rows columns value for each non zero elements:\n");
for(i=1;i≤s[0][2];i++){
 scanf("%d %d %d",&s[i][0],&s[i][1],&s[i][2]);
 }
t[0][0]=s[0][1];
t[0][1]=s[0][0];
t[0][2]=s[0][2];
for(i=0;i<s[0][1];i++){
 for(j=1; j ≤s[0][2]; j++){
  if(s[j][1]==i){
  t[k][0]=s[j][1];
  t[k][1]=s[j][0];
  t[k][2]=s[j][2];
  k++;
  }
 }
}
printf("\n transpose of sparce matrix:\n");
printf("row\tcolumn\tvalue\n");
for(i=0;i≤t[0][2];i++){
 printf("%d\t%d\t%d\n",t[i][0],t[i][1],t[i][2]);
}
return 0;
}
```

PROGRAM OUTPUT

Execution Verification

```
Enter total number of rows and columns for the matrix: 3 3
Enter number of non-zero elements for the matrix: 3

Enter Triplets for the Matrix (Row, Column, Value):
0 1 5
1 2 8
2 0 3

The transposed matrix is:
3 3 3
0 2 3
1 0 5
2 1 8
```

PROGRAM 6 6.c Stack Operations Using Array

C SOURCE CODE 6.c

```c
#include <stdio.h>
#define MAX 5
int stack[MAX];
int top= -1;
void push() {
int item;
if(top==MAX-1)
{
 printf("\n error 404 \n");
 printf("stack overflow \n");
 }
else{
 printf("enter elements:");
 scanf("%d",&item);
 top++;
 stack[top]=item;
 }
}

void pop() {
if(top==-1) {
 printf("\n error 404\n");
 printf("stack underflow\n");
 }
else {
 printf("deleted element:%d\n",stack[top]);
 top--;
 }
}

void display() {
int i;
if(top==-1){
 printf("\n error 404\n");
 printf("stack is empty\n");
 }
else {
 printf("stack elements are:\n");
  for(i=top;i≥0;i--){
   printf("%d\n",stack[i]);
   }
  }
 }
 int main() {
 int choice;
 do{
  printf("\n 1.push \n 2.pop \n 3.display \n 4.exit \n ");
  printf("enter your choice:");
  scanf("%d",&choice);
  switch(choice){
   case 1:
     push();
     break;
   case 2:
     pop();
     break;
   case 3:
     display();
     break;
   case 4:
     printf("you have succesfully exited \n ");
     break;
    default:
   printf("invalid choice \n ");}

```

```
    } while (choice≠4);

    return 0;
}
```

PROGRAM OUTPUT Execution Verification

```
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 1
Enter value to push: 10
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 1
Enter value to push: 20
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 3
Displaying contents:
10
20
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 2
Popped value: 20
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 3
Displaying contents:
10
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 4
Exiting ...
```

PROGRAM 7 7.c Linear Queue Operations Using Array

C SOURCE CODE 7.c

```c
#include <stdio.h>
#define MAX 5

int queue[MAX];
int front= -1,rear= -1;

 void enqueue() {
 int item;
 if(rear==MAX-1){
  printf("queue is overflow\n"); }
 else {
  if (front==-1)
   front=0;
   printf("enter the element:");
   scanf("%d",&item);
   rear++;
   queue[rear]=item;
   }
  }
 void dequeue() {
  if(front== -1|| front>rear){
   printf("queue underflow\n"); }
  else {
   printf("deleted element:%d\n",queue[front]);
   front++;
   if(front>rear)
    front=rear=-1;
   }
 }
 void display() {
 int i;
 if(front== -1){
  printf("queue is empty\n"); }
 else {
  printf("queue elements are:\n");
 for(i=front;i≤rear;i++){
  printf("%d\n",queue[i]);
  }
 }
}
int main() {
int choice;
 do{
  printf("\n1.enqueue \n 2.dequeue \n 3.display \n 4.exit \n");
  printf("enter your choice:");
  scanf("%d",&choice);
  switch (choice){
   case 1:
    enqueue();
    break;
   case 2:
    dequeue();
    break;
   case 3:
    display();
    break;
   case 4:
    printf("you have sucessfully exited\n");
    break;
   default:
   printf("invalid choice\n"); }
  } while(choice≠4);
  return 0;
}
```

```
65
```

PROGRAM OUTPUT Execution Verification

```
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to enqueue: 100
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to enqueue: 200
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 3
Displaying Contents:
100
200
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 2
Dequeued value: 100
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 3
Displaying Contents:
200
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 4
Exiting ...
```

PROGRAM 8 8.c

Circular Queue Operations Using Array

C SOURCE CODE 8.c

```c
#include <stdio.h>
#define MAX 5

int queue[MAX];
int front= -1,rear= -1, count=0;
 // enqueue
 void enqueue() {
 int item;
 if(count==MAX){
  printf("circular queue is overflow\n"); }
 else {
  if (front==-1)
   front=0;
   printf("enter the element:");
   scanf("%d",&item);
   rear=(rear+1)% MAX;
   count++;
   queue[rear]=item;
   }
  }
   //dequeue
 void dequeue() {
  if(count==0){
   printf("circular queue underflow\n"); }
  else {
   printf("deleted element:%d\n",queue[front]);
   front=(front+1)% MAX;
   count--;
 } }
 // diplay
 void display() {
 int i;
 if(count==0){
  printf("circular queue is empty\n"); }
 else {
  printf("queue elements are:\n");
 for(i=front;i≤rear;i++){
  printf("%d\n",queue[i]);
  }
 }
}
//peak of the last element
int peek() {
    if (count == 0) {
        printf("queue is empty\n"); }
 else {
  printf("peeked value: %d\n", queue[front]);
    }
}
int main() {
int choice;
 do{
  printf("\n1.enqueue \n 2.dequeue \n 3.peek \n 4.display \n 5.exit \n");
  printf("enter your choice:");
  scanf("%d",&choice);
  switch (choice){
   case 1:
    enqueue();
    break;
   case 2:
    dequeue();
    break;
   case 3:
    peek();
    break;
```

```
    case 4:
     display();
     break;
    case 5:
     printf("you have sucessfully exited\n");
     break;
    default:
    printf("invalid choice\n"); }
   } while(choice≠5);
   return 0;
}
```

PROGRAM OUTPUT Execution Verification

```
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to enqueue: 10
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to enqueue: 20
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to enqueue: 30
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 3
Displaying Contents:
10
20
30
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 2
Dequeued value: 10
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to enqueue: 40
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 3
Displaying Contents:
20
30
40
1. Enqueue
2. Dequeue
3. Display
4. Exit
```

```
Enter your choice: 4
Exiting ...
```

PROGRAM 9 9.c

Double Ended Queue (Deque) Operations

C SOURCE CODE 9.c

```c
#include <stdio.h>
#define MAX 5

int deque[MAX];
int front=-1,rear=-1;
// TO PUSH THE VALUE INTO DEQUEE
void push() {
int item;
if((front==0 && rear==MAX-1)||(front==rear+1))
{
 printf("deque is full\n");
 return;
}
printf("enter element:");
scanf("%d",&item);
if (front==-1)
 front=rear=0;
 else if (front==0)
 front=MAX-1;
 else
  front--;
 deque[front]=item;
}
// TO INJECT THE VALUE INTO DEQUEE

void inject () {
int item;
if((front==0 && rear==MAX-1)||(front==rear+1))
{
 printf("deque is full\n");
 return;
}
printf("enter element:");
scanf("%d",&item);
if (front==-1)
 front=rear=0;
 else if(rear== MAX-1)
  rear=0;
 else
 rear++;
deque[rear]=item;
}
// TO POP THE VALUE INTO DEQUEE

void pop() {
if (front==-1) {
 printf("deque is empty\n");
 return;
}
printf("deleted element:%d \n",deque[front]);
if (front==rear)
 front=rear=-1;
else if (front== MAX-1 )
 front=0;
else
 front++;
}
// TO EJECT THE VALUE INTO DEQUEE

void eject () {
if (front==-1) {
 printf("deque is empty\n");
 return;
}
printf("deleted element:%d \n",deque[rear]);
```

```c
if (front==rear)
front=rear=-1;
else if (rear==0)
 front= MAX-1;
else
 rear--;
}
// TO DISPLAY THE VALUE INTO DEQUEE

void display () {
int i;
if(front==-1) {
printf("deque is empty\n");
return;
}
printf("deque elements are:\n");
i=front;
while(1)
{
 printf("%d\t",deque[i]);
 if(i==rear)
 break;
 i=(i+1)% MAX;
}
printf("\n");
printf("\t");
}

int main () {
int choice;
while(1)
{
printf("\n  DEQUE  \n");
printf("1.push \n");
printf("2.inject \n");
printf("3.pop\n");
printf("4.eject\n");
printf("5.display\n");
printf("6.exit\n");
printf("enter your choice:");
scanf("%d",&choice);
switch (choice) {
case 1:
 push();
 break;
case 2:
 inject();
 break;
case 3:
 pop();
 break;
case 4:
 eject();
 break;
case 5:
 display();
 break;
case 6:
 return 0;
 default:
printf("invalid choice\n");
}
}
}
```

PROGRAM OUTPUT Execution Verification

```
1. Insert Front
2. Insert Rear
3. Remove Front
```

```
4. Remove Rear
5. Display
6. Exit
Enter your choice: 2
Enter value to insert at rear: 10
1. Insert Front
2. Insert Rear
3. Remove Front
4. Remove Rear
5. Display
6. Exit
Enter your choice: 2
Enter value to insert at rear: 20
1. Insert Front
2. Insert Rear
3. Remove Front
4. Remove Rear
5. Display
6. Exit
Enter your choice: 1
Enter value to insert at front: 5
1. Insert Front
2. Insert Rear
3. Remove Front
4. Remove Rear
5. Display
6. Exit
Enter your choice: 5
Displaying Contents:
5
10
20
1. Insert Front
2. Insert Rear
3. Remove Front
4. Remove Rear
5. Display
6. Exit
Enter your choice: 3
Removed from front: 5
1. Insert Front
2. Insert Rear
3. Remove Front
4. Remove Rear
5. Display
6. Exit
Enter your choice: 4
Removed from rear: 20
1. Insert Front
2. Insert Rear
3. Remove Front
4. Remove Rear
5. Display
6. Exit
Enter your choice: 5
Displaying Contents:
10
1. Insert Front
2. Insert Rear
3. Remove Front
4. Remove Rear
5. Display
6. Exit
Enter your choice: 6
Exiting ...
```

PROGRAM 10 10.c

Infix to Postfix Expression Conversion

C SOURCE CODE 10.c

```c
#include<stdio.h>
#include<ctype.h>
char stack[100];
int top=-1;
void push(char x)
{
stack[++top]=x;
}
char pop()
{
if(top==-1)
return-1;
return stack[top--];
}
int priority(char x)
{
if(x=='(')
return 0;
if(x=='+'||x=='-')
return 1;
if(x=='*'||x=='/')
return 2;
if(x=='^')
return 3;

return 0;
}
int main()
{
char exp[100];
char x;

int i=0;
printf("enter the infix representation:");
scanf("%s",exp);
printf("postfix expression:");
while(exp[i]≠'\0')
{
if(isalnum(exp[i]))
{
printf("%c",exp[i]);
}
else if(exp[i]=='(')
{
push(exp[i]);
}
else if(exp[i]==')')
{
while(top≠-1 && (x=pop())≠'(')
{
printf("%c",x);
}
}
else
{
while(top≠-1 && stack[top]≠'(' && priority(stack[top])≥priority(exp[i]) && exp[i]≠'^')
{
printf("%c",pop());
}
push(exp[i]);
}
i++;
}
while(top≠-1){
printf("%c",pop());
```

```
66  }
67  printf("\n");
68  return 0;
69  }
70
```

PROGRAM OUTPUT Execution Verification

```
Enter infix expression: (2+3)*(4-5)
Postfix Expression:
2 3 + 4 5 - *
```

PROGRAM 11 11.c

Evaluation of Postfix Expression Using Stack

C SOURCE CODE 11.c

```c
#include<stdio.h>
#include<ctype.h>
#include<math.h>
#define MAX 100
int stack[MAX];
int top=-1;
void push(int x)
{
stack[++top]=x;
}
int pop()
{
return stack[top--];
}
int main()
{
char postfix[MAX];
int i,a,b,result;
printf("Enter the postfix expression: \n");
scanf("%s",postfix);
for(i=0;postfix[i]≠'\0';i++)
{
if (isdigit(postfix[i]))
{
push(postfix[i]-'0');
}
else{
b=pop();
a=pop();
switch(postfix[i])
{
case'+':
result=a+b;
break;

case'-':
result=a-b;
break;

case'*':
result=a*b;
break;

case'/':
result=a/b;
break;

case'%':
result=a%b;
break;

case'^':
result=a^b;
break;
}
push(result);
}}
printf("Result: %d\n",pop());
return 0;
}

```

PROGRAM OUTPUT Execution Verification

```
Enter the postfix expression: 2 3 + 4 5 - *
Result: -5
```

PROGRAM 12 `12.c` **Singly Linked List Operations**

C SOURCE CODE 12.c

```c
#include <stdio.h>
#include <stdlib.h>
struct Node {
 int data;
 struct Node*next;
};
struct Node*head=NULL;

void insertAtEnd(int value) {
struct Node *newNode=(struct Node*) malloc(sizeof(struct Node));
newNode->data=value;
newNode->next=NULL;
if(head==NULL) {
 head= newNode;
 }
else {
 struct Node *temp=head;
 while (temp->next!=NULL) {
 temp=temp->next;
 }
temp->next=newNode;
}
}

void insertAtBeginning(int value) {
struct Node *newNode=(struct Node*) malloc(sizeof(struct Node));
newNode->data=value;
newNode->next=head;
head=newNode;
}

void insertAtPosition(int value,int position) {
struct Node *newNode=(struct Node*) malloc(sizeof(struct Node));
newNode->data=value;
newNode->next=NULL;
if (position==1) {
newNode->next=head;
head=newNode;
return;
}
struct Node *temp=head;
for(int i=1;i<position-1;i++) {
if(temp==NULL) {
 printf("invalid position:\n");
 return;
}
temp=temp->next;
temp->next=newNode;
}
}
void deleteAtBeginning() {
if (head==NULL) {
 printf("list is empty:\n");
 return;
}
struct Node *temp=head;
head=head->next;
free(temp);
}

void deleteAtEnd() {
if (head==NULL) {
 printf("list is empty:\n");
 return;
}
```

```c
if (head->next==NULL){
 free(head);
 head=NULL;
 return;
}
struct Node *temp=head;
while(temp->next->next!=NULL){
temp=temp->next;
}
free(temp->next);
temp->next=NULL;
}

void deleteAtPosition() {
int position;
if (head==NULL) {
 printf("list is empty:\n");
 return;
 }
if (position==1) {
struct Node *temp=head;
head=head->next;
free(temp);
return;
}
struct Node *temp=head;
for(int i=1;i<position-1;i++) {
if(temp==NULL) {
 printf("invalid position:\n");
 return;
}
temp=temp->next;
}
if(temp->next==NULL) {
printf("invalid position! \n");
return;
}
struct Node *deleteNode=temp->next;
temp->next=deleteNode->next;
free(deleteNode);
}

void display() {
if (head==NULL) {
printf("list is empty! \n");
return;
}
struct Node *temp=head;
printf("linked list:");
while (temp!=NULL) {
printf("%d->",temp->data);
temp=temp->next;
}
printf("NULL\n");
}

int main() {
int choice,value,position;
while(1) {
printf("\n singly linked list operation   \n");
printf("\n1.insert at End\n");
printf("\n2.insert at Beginning\n");
printf("\n3.insert at Position\n");
printf("\n4.delete at Beginning\n");
printf("\n5.delete at End\n");
printf("\n6.delete at Position\n");
printf("\n7.display list\n");
printf("\n8.exit\n");
printf("enter your choice:");
scanf("%d",&choice);
```

```c
switch (choice) {
 case 1:
    printf("enter value to insert at End:");
    scanf("%d",&value);
    insertAtEnd(value);
    break;
 case 2:
    printf("enter value to insert at Beginning:");
    scanf("%d",&value);
    insertAtBeginning(value);
    break;
 case 3:
    printf("enter value to insert:");
    scanf("%d",&value);
    printf("enter position:");
    scanf("%d",&position);
    insertAtPosition(value,position);
    break;
 case 4:
    deleteAtBeginning();
    break;
 case 5:
    deleteAtEnd();
    break;
 case 6:
    printf("enter position to delete:");
    scanf("%d",&position);
    deleteAtPosition(position);
    break;
 case 7:
    display();
    break;
 case 8:
    printf("exiting  \n");
    exit(0);
    default:
    printf("invalid choice!try again\n");
  }
 }
return 0;
}
```

PROGRAM OUTPUT Execution Verification

```
- - - Linked List Menu - - -
1. Create
2. Display
3. Search
4. Insert at Beginning
5. Insert at Middle
6. Insert at End
7. Delete at Beginning
8. Delete at Middle
9. Delete at End
10. Exit
Enter choice : 1
Enter data : 10
Node created successfully

- - - Linked List Menu - - -
Enter choice : 2
Linked List: 10 --> NULL

- - - Linked List Menu - - -
Enter choice : 4
Enter data : 5
Node inserted successfully at beginning

- - - Linked List Menu - - -
```

```
Enter choice : 6
Enter data : 30
Node inserted successfully at end

- - - Linked List Menu - - -
Enter choice : 5
Enter data : 20
Enter position : 3
Node inserted successfully at position 3

- - - Linked List Menu - - -
Enter choice : 2
Linked List: 5 → 10 → 20 → 30 → NULL

- - - Linked List Menu - - -
Enter choice : 3
Enter element to search : 20
Element 20 found at position 3

- - - Linked List Menu - - -
Enter choice : 8
Enter position to delete : 3
Deleted element: 20

- - - Linked List Menu - - -
Enter choice : 7
Deleted element: 5

- - - Linked List Menu - - -
Enter choice : 9
Deleted element: 30

- - - Linked List Menu - - -
Enter choice : 2
Linked List: 10 → NULL

- - - Linked List Menu - - -
Enter choice : 10
Exiting ...
```

PROGRAM 13 13.c

Doubly Linked List Operations

C SOURCE CODE 13.c

```c
/* Implementation of Stack using Linked List in C */

#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *top = NULL;

void push() {
    int val;
    printf("Enter value: ");
    scanf("%d", &val);

    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    newNode->data = val;
    newNode->next = top;

    if (top != NULL) {
        newNode->next= top;
    }

    top = newNode;
}

int pop() {
    if (top == NULL) {
        printf("Stack Underflow\n");
        return -1;
    }
    struct Node *temp = top;
    top = top->next;
    printf("Popped element: %d\n", temp->data);
    free(temp);
}

void display() {
    if (top == NULL) {
        printf("Stack Underflow\n");
        return;
    }
    printf("Displaying contents:\n");
    printf("NULL\n");
    struct Node *temp = top;
    while (temp != NULL) {

        printf("%d\n ", temp->data);
        temp = temp->next;
    }
}

void peek(){
    if(top == NULL) {
        printf("Stack Underflow\n");
        return;
    }
    printf("Displaying contents:\n");
    struct Node *temp = top;
    if (temp != NULL) {
        printf("%d\n", temp->data);
    }
}
```

```c


int main() {
    int choice, value;

    do {
        printf("1. Push\n");
        printf("2. Pop\n");
        printf("3. Display\n");
        printf("4. PEEK\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                push();
                break;
            case 2:
                pop();
                break;
            case 3:
                display();
                break;
            case 4:
                peek();
                break;
            case 5:
                printf("Exiting ... \n");
                break;
            default:
                printf("Invalid choice. Please try again.\n");
        }
    } while (choice ≠ 5);

    return 0;
}
```

PROGRAM OUTPUT Execution Verification

```
- - - Doubly Linked List Menu - - -
1. Create
2. Display
3. Search
4. Insert at Beginning
5. Insert at Middle
6. Insert at End
7. Delete at Beginning
8. Delete at Middle
9. Delete at End
10. Exit
Enter choice : 1
Enter data : 10
Node created successfully

- - - Doubly Linked List Menu - - -
Enter choice : 4
Enter data : 5
Node inserted successfully at beginning

- - - Doubly Linked List Menu - - -
Enter choice : 6
Enter data : 30
Node inserted successfully at end

- - - Doubly Linked List Menu - - -
Enter choice : 5
Enter data : 20
Enter position : 3
```

```
Node inserted successfully at position 3

- - - Doubly Linked List Menu - - -
Enter choice : 2
Linked List: 5 ⟷ 10 ⟷ 20 ⟷ 30 ⟷ NULL

- - - Doubly Linked List Menu - - -
Enter choice : 3
Enter element to search : 20
Element 20 found at position 3

- - - Doubly Linked List Menu - - -
Enter choice : 8
Enter position to delete : 3
Deleted element: 20

- - - Doubly Linked List Menu - - -
Enter choice : 7
Deleted element: 5

- - - Doubly Linked List Menu - - -
Enter choice : 9
Deleted element: 30

- - - Doubly Linked List Menu - - -
Enter choice : 2
Linked List: 10 ⟷ NULL

- - - Doubly Linked List Menu - - -
Enter choice : 10
Exiting ...
```

PROGRAM 14 14.c

Stack Operations Using Linked List

C SOURCE CODE 14.c

```c
/* Implementation of Queue using Linked List in C */

#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *front = NULL;
struct Node *rear = NULL;

void enqueue() {
    int val;
    printf("\n Enter value to enqueue: ");
    scanf("%d", &val);
    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));

    newNode->data = val;
    newNode->next = NULL;

    if (front == NULL && rear == NULL) {
        front = rear = newNode;
    } else {
        rear->next = newNode;
        rear = newNode;
    }
}

void dequeue() {
    if (front == NULL && rear == NULL) {
        printf("Queue Underflow\n");
        return;
    }
    struct Node *temp = front;
    int val = temp->data;
    front = front->next;
    printf("Dequeued: %d\n", val);
    free(temp);
    if (front == NULL) rear = NULL;
}

void display() {
    if (front == NULL && rear == NULL) {
        printf("Queue Empty\n");
        return;
    }
    printf("Displaying Contents:\n");
    printf("NULL\t");

    struct Node *temp = front;
    while (temp != NULL) {
        printf("%d\t", temp->data);
        temp = temp->next;
    }
    printf("\n");
}
void peek() {
    if (front == NULL && rear == NULL) {
        printf("Queue Empty\n");
        return;
    }
    printf("Displaying last Contents:\n");
    struct Node *temp = front;
```

```c
    if(temp ≠ NULL) {
        printf("%d\n", temp→data);
    }
}

int main() {
    int choice, value;

    do {
        printf("1. Enqueue\n");
        printf("2. Dequeue\n");
        printf("3. Display\n");
        printf("4. PEEK\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                enqueue();
                break;
            case 2:
                dequeue();
                break;
            case 3:
                display();
                break;
            case 4:
                peek();
                break;
            case 5:
                printf("Exiting ... \n");
                break;
            default:
                printf("Invalid choice! Please try again.\n");
        }
    } while (choice ≠ 5);

    return 0;
}
```

PROGRAM OUTPUT Execution Verification

```
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 1
Enter value to push: 10
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 1
Enter value to push: 20
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 3
Displaying contents:
20
10
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 2
```

```
Popped value: 20
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 3
Displaying contents:
10
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 4
Exiting ...
```

PROGRAM 15 15.c

Queue Operations Using Linked List

C SOURCE CODE

15.c

```c
#include<stdio.h>
#include<stdlib.h>
void main()
{
    int arr[100],key=0,found,i,n;
    int t;
    printf("enter no of elements");
    scanf("%d",&n);
    for(i=0;i<n;i++)
    {
    printf("enter elements");
    scanf("%d",&arr[i]);
    }
    // display the elements
    printf("DISPLAY \n");
    for(i=0;i<n;i++)
    {
    printf("%d element :%d\t",i,arr[i]);
    printf("\t");
    }
    printf("\n");

    printf("enter the element to be searched");
    scanf("%d",&found);
    for(i=0;i<n;i++)
    {
        if(found==arr[i])
        {
            key=1;
            t=i;
        }
    }
    if(key==0)
    {
        printf("element is not on the array");

    }
    else
    {
        printf("element found at%d",t);
    }

}
```

PROGRAM OUTPUT

Execution Verification

```
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to enqueue: 100
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to enqueue: 200
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 3
Displaying Contents:
```

```
100
200
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 2
Dequeued value: 100
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 3
Displaying Contents:
200
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 4
Exiting...
```

PROGRAM 12 12.c

Singly Linked List Operations

C SOURCE CODE 12.c

```c
#include <stdio.h>
#include <stdlib.h>
struct Node {
 int data;
 struct Node*next;
};
struct Node*head=NULL;

void insertAtEnd(int value) {
struct Node *newNode=(struct Node*) malloc(sizeof(struct Node));
newNode->data=value;
newNode->next=NULL;
if(head==NULL) {
 head= newNode;
 }
else {
 struct Node *temp=head;
 while (temp->next!=NULL) {
 temp=temp->next;
 }
temp->next=newNode;
}
}

void insertAtBeginning(int value) {
struct Node *newNode=(struct Node*) malloc(sizeof(struct Node));
newNode->data=value;
newNode->next=head;
head=newNode;
}

void insertAtPosition(int value,int position) {
struct Node *newNode=(struct Node*) malloc(sizeof(struct Node));
newNode->data=value;
newNode->next=NULL;
if (position==1) {
newNode->next=head;
head=newNode;
return;
}
struct Node *temp=head;
for(int i=1;i<position-1;i++) {
if(temp==NULL) {
 printf("invalid position:\n");
 return;
}
temp=temp->next;
temp->next=newNode;
}
}
void deleteAtBeginning() {
if (head==NULL) {
 printf("list is empty:\n");
 return;
}
struct Node *temp=head;
head=head->next;
free(temp);
}

void deleteAtEnd() {
if (head==NULL) {
 printf("list is empty:\n");
 return;
}
```

```c
if (head->next==NULL){
 free(head);
 head=NULL;
 return;
}
struct Node *temp=head;
while(temp->next->next!=NULL){
temp=temp->next;
}
free(temp->next);
temp->next=NULL;
}

void deleteAtPosition() {
int position;
if (head==NULL) {
 printf("list is empty:\n");
 return;
 }
if (position==1) {
struct Node *temp=head;
head=head->next;
free(temp);
return;
}
struct Node *temp=head;
for(int i=1;i<position-1;i++) {
if(temp==NULL) {
 printf("invalid position:\n");
 return;
}
temp=temp->next;
}
if(temp->next==NULL) {
printf("invalid position! \n");
return;
}
struct Node *deleteNode=temp->next;
temp->next=deleteNode->next;
free(deleteNode);
}

void display() {
if (head==NULL) {
printf("list is empty! \n");
return;
}
struct Node *temp=head;
printf("linked list:");
while (temp!=NULL) {
printf("%d->",temp->data);
temp=temp->next;
}
printf("NULL\n");
}

int main() {
int choice,value,position;
while(1) {
printf("\n singly linked list operation   \n");
printf("\n1.insert at End\n");
printf("\n2.insert at Beginning\n");
printf("\n3.insert at Position\n");
printf("\n4.delete at Beginning\n");
printf("\n5.delete at End\n");
printf("\n6.delete at Position\n");
printf("\n7.display list\n");
printf("\n8.exit\n");
printf("enter your choice:");
scanf("%d",&choice);
```

```c
switch (choice) {
 case 1:
    printf("enter value to insert at End:");
    scanf("%d",&value);
    insertAtEnd(value);
    break;
 case 2:
    printf("enter value to insert at Beginning:");
    scanf("%d",&value);
    insertAtBeginning(value);
    break;
 case 3:
    printf("enter value to insert:");
    scanf("%d",&value);
    printf("enter position:");
    scanf("%d",&position);
    insertAtPosition(value,position);
    break;
 case 4:
    deleteAtBeginning();
    break;
 case 5:
    deleteAtEnd();
    break;
 case 6:
    printf("enter position to delete:");
    scanf("%d",&position);
    deleteAtPosition(position);
    break;
 case 7:
    display();
    break;
 case 8:
    printf("exiting  \n");
    exit(0);
    default:
    printf("invalid choice!try again\n");
  }
 }
return 0;
}
```

PROGRAM OUTPUT Execution Verification

```
- - - Linked List Menu - - -
1. Create
2. Display
3. Search
4. Insert at Beginning
5. Insert at Middle
6. Insert at End
7. Delete at Beginning
8. Delete at Middle
9. Delete at End
10. Exit
Enter choice : 1
Enter data : 10
Node created successfully

- - - Linked List Menu - - -
Enter choice : 4
Enter data : 5
Node inserted successfully at beginning

- - - Linked List Menu - - -
Enter choice : 6
Enter data : 30
Node inserted successfully at end
```

```
- - - Linked List Menu - - -
Enter choice : 5
Enter data : 20
Enter position : 3
Node inserted successfully at position 3

- - - Linked List Menu - - -
Enter choice : 2
Linked List: 5 → 10 → 20 → 30 → NULL

- - - Linked List Menu - - -
Enter choice : 3
Enter element to search : 20
Element 20 found at position 3

- - - Linked List Menu - - -
Enter choice : 8
Enter position to delete : 3
Deleted element: 20

- - - Linked List Menu - - -
Enter choice : 7
Deleted element: 5

- - - Linked List Menu - - -
Enter choice : 9
Deleted element: 30

- - - Linked List Menu - - -
Enter choice : 2
Linked List: 10 → NULL

- - - Linked List Menu - - -
Enter choice : 10
Exiting ...
```

PROGRAM 13 13.c

Doubly Linked List Operations

C SOURCE CODE 13.c

```c
/* Implementation of Stack using Linked List in C */

#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *top = NULL;

void push() {
    int val;
    printf("Enter value: ");
    scanf("%d", &val);

    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    newNode->data = val;
    newNode->next = top;

    if (top != NULL) {
        newNode->next= top;
    }

    top = newNode;
}

int pop() {
    if (top == NULL) {
        printf("Stack Underflow\n");
        return -1;
    }
    struct Node *temp = top;
    top = top->next;
    printf("Popped element: %d\n", temp->data);
    free(temp);
}

void display() {
    if (top == NULL) {
        printf("Stack Underflow\n");
        return;
    }
    printf("Displaying contents:\n");
    printf("NULL\n");
    struct Node *temp = top;
    while (temp != NULL) {

        printf("%d\n ", temp->data);
        temp = temp->next;
    }
}

void peek(){
    if(top == NULL) {
        printf("Stack Underflow\n");
        return;
    }
    printf("Displaying contents:\n");
    struct Node *temp = top;
    if (temp != NULL) {
        printf("%d\n", temp->data);
    }
}
```

```c


int main() {
    int choice, value;

    do {
        printf("1. Push\n");
        printf("2. Pop\n");
        printf("3. Display\n");
        printf("4. PEEK\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                push();
                break;
            case 2:
                pop();
                break;
            case 3:
                display();
                break;
            case 4:
                peek();
                break;
            case 5:
                printf("Exiting ... \n");
                break;
            default:
                printf("Invalid choice. Please try again.\n");
        }
    } while (choice ≠ 5);

    return 0;
}
```

PROGRAM OUTPUT Execution Verification

```
- - - Doubly Linked List Menu - - -
1. Create
2. Display
3. Search
4. Insert at Beginning
5. Insert at Middle
6. Insert at End
7. Delete at Beginning
8. Delete at Middle
9. Delete at End
10. Exit
Enter choice : 1
Enter data : 10
Node created successfully

- - - Doubly Linked List Menu - - -
Enter choice : 4
Enter data : 5
Node inserted successfully at beginning

- - - Doubly Linked List Menu - - -
Enter choice : 6
Enter data : 30
Node inserted successfully at end

- - - Doubly Linked List Menu - - -
Enter choice : 5
Enter data : 20
Enter position : 3
```

```
Node inserted successfully at position 3

- - - Doubly Linked List Menu - - -
Enter choice : 2
Linked List: 5 ⟷ 10 ⟷ 20 ⟷ 30 ⟷ NULL

- - - Doubly Linked List Menu - - -
Enter choice : 3
Enter element to search : 20
Element 20 found at position 3

- - - Doubly Linked List Menu - - -
Enter choice : 8
Enter position to delete : 3
Deleted element: 20

- - - Doubly Linked List Menu - - -
Enter choice : 7
Deleted element: 5

- - - Doubly Linked List Menu - - -
Enter choice : 9
Deleted element: 30

- - - Doubly Linked List Menu - - -
Enter choice : 2
Linked List: 10 ⟷ NULL

- - - Doubly Linked List Menu - - -
Enter choice : 10
Exiting ...
```

PROGRAM 14 14.c Stack Operations Using Linked List

C SOURCE CODE 14.c

```c
/* Implementation of Queue using Linked List in C */

#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *front = NULL;
struct Node *rear = NULL;

void enqueue() {
    int val;
    printf("\n Enter value to enqueue: ");
    scanf("%d", &val);
    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));

    newNode->data = val;
    newNode->next = NULL;

    if (front == NULL && rear == NULL) {
        front = rear = newNode;
    } else {
        rear->next = newNode;
        rear = newNode;
    }
}

void dequeue() {
    if (front == NULL && rear == NULL) {
        printf("Queue Underflow\n");
        return;
    }
    struct Node *temp = front;
    int val = temp->data;
    front = front->next;
    printf("Dequeued: %d\n", val);
    free(temp);
    if (front == NULL) rear = NULL;
}

void display() {
    if (front == NULL && rear == NULL) {
        printf("Queue Empty\n");
        return;
    }
    printf("Displaying Contents:\n");
    printf("NULL\t");

    struct Node *temp = front;
    while (temp != NULL) {
        printf("%d\t", temp->data);
        temp = temp->next;
    }
    printf("\n");
}
void peek() {
    if (front == NULL && rear == NULL) {
        printf("Queue Empty\n");
        return;
    }
    printf("Displaying last Contents:\n");
    struct Node *temp = front;
```

```c
    if(temp ≠ NULL) {
        printf("%d\n", temp→data);
    }
}

int main() {
    int choice, value;

    do {
        printf("1. Enqueue\n");
        printf("2. Dequeue\n");
        printf("3. Display\n");
        printf("4. PEEK\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                enqueue();
                break;
            case 2:
                dequeue();
                break;
            case 3:
                display();
                break;
            case 4:
                peek();
                break;
            case 5:
                printf("Exiting ... \n");
                break;
            default:
                printf("Invalid choice! Please try again.\n");
        }
    } while (choice ≠ 5);

    return 0;
}
```

PROGRAM OUTPUT Execution Verification

```
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 1
Enter value to push: 10
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 1
Enter value to push: 20
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 3
Displaying contents:
20
10
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 2
```

```
Popped value: 20
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 3
Displaying contents:
10
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 4
Exiting ...
```

PROGRAM 15 15.c

Queue Operations Using Linked List

C SOURCE CODE

15.c

```c
#include<stdio.h>
#include<stdlib.h>
void main()
{
    int arr[100],key=0,found,i,n;
    int t;
    printf("enter no of elements");
    scanf("%d",&n);
    for(i=0;i<n;i++)
    {
    printf("enter elements");
    scanf("%d",&arr[i]);
    }
    // display the elements
    printf("DISPLAY \n");
    for(i=0;i<n;i++)
    {
    printf("%d element :%d\t",i,arr[i]);
    printf("\t");
    }
    printf("\n");

    printf("enter the element to be searched");
    scanf("%d",&found);
    for(i=0;i<n;i++)
    {
        if(found==arr[i])
        {
            key=1;
            t=i;
        }
    }
    if(key==0)
    {
        printf("element is not on the array");

    }
    else
    {
        printf("element found at%d",t);
    }

}
```

PROGRAM OUTPUT

Execution Verification

```
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to enqueue: 100
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to enqueue: 200
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 3
Displaying Contents:
```

```
100
200
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 2
Dequeued value: 100
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 3
Displaying Contents:
200
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 4
Exiting...
```

PROGRAM 16 `16_linear_search.c`

Linear Search

C SOURCE CODE 16_linear_search.c

```c
#include <stdio.h>

int main(void) {
    int size;
    int target;
    int found = 0;

    printf("Enter the size of the array: ");
    scanf("%d", &size);

    int numbers[size];

    printf("Enter %d numbers:\n", size);
    for (int i = 0; i < size; i++) {
        scanf("%d", &numbers[i]);
    }

    printf("Enter a number to search: ");
    scanf("%d", &target);

    // Check each element one by one.
    for (int i = 0; i < size; i++) {
        if (numbers[i] == target) {
            printf("Number found at index %d.\n", i);
            found = 1;
            break;
        }
    }

    if (!found) {
        printf("Number not found.\n");
    }

    return 0;
}
```

PROGRAM OUTPUT Execution Verification

```
Enter the size of the array: 5\nEnter 5 numbers:\n10 20 30 40 50\nEnter a number to search: 30\nNumber fo
und at index 2.
```

PROGRAM 17 `17_binary_search.c`

Binary Search

C SOURCE CODE 17_binary_search.c

```c
#include <stdio.h>

int main() {
        int numbers[100];
        int n, i, target;
        int left, right, middle;
        int found = 0;

        printf("Enter the number of elements: ");
        scanf("%d", &n);

        printf("Enter %d elements in sorted order: ", n);
        for (i = 0; i < n; i++) {
                scanf("%d", &numbers[i]);
        }

        printf("Enter the number to search: ");
        scanf("%d", &target);

        left = 0;
        right = n - 1;

        while (left ≤ right) {
                middle = (left + right) / 2;
        printf("middle index = %d\tvalue = %d\n", middle, numbers[middle]);

                if (numbers[middle] == target) {
                        printf("Number found at position %d\n", middle + 1);
                        found = 1;
                        break;
                } else if (numbers[middle] < target) {
                        left = middle + 1;
                } else {
                        right = middle - 1;
                }
        }

        if (found == 0) {
                printf("Number not found\n");
        }

        return 0;
}

```

PROGRAM OUTPUT Execution Verification

```
Enter the size of the array: 5\nEnter 5 sorted numbers:\n10 20 30 40 50\nEnter a number to search: 40\nNu
mber found at index 3.
```

PROGRAM 18 `18_insertion_sort.c` **Insertion Sort**

C SOURCE CODE 18_insertion_sort.c

```c
#include <stdio.h>

int main(void)
{
    int numbers[100];
    int n, i, j, key;

    printf("Enter the number of elements: ");
    scanf("%d", &n);

    printf("Enter the elements:\n");
    for (i = 0; i < n; i++)
    {
        scanf("%d", &numbers[i]);
    }

    printf("\nInitial array: ");
    for (i = 0; i < n; i++)
    {
        printf("%d ", numbers[i]);
    }

    for (i = 1; i < n; i++)
    {
        key = numbers[i];
        j = i - 1;

        while (j ≥ 0 && numbers[j] > key)
        {
            numbers[j + 1] = numbers[j];
            j--;
        }

        numbers[j + 1] = key;

        printf("\nStep %d: ", i);
        for (j = 0; j < n; j++)
        {
            printf("%d ", numbers[j]);
        }
    }

    printf("\n\nSorted array: ");
    for (i = 0; i < n; i++)
    {
        printf("%d ", numbers[i]);
    }

    printf("\n");
    return 0;
}
```

PROGRAM OUTPUT Execution Verification

```
Enter number of elements: 5\nEnter 5 integers:\n50 20 40 10 30\nSorted list in ascending order:\n10 20 30 40 50
```

PROGRAM 19 `19_selection_sort.c`

Selection Sort

C SOURCE CODE 19_selection_sort.c

```c
#include <stdio.h>

int main()
{
        int a[100];
        int n, i, j, smallest, temp;

        printf("Enter the number of elements: ");
        scanf("%d", &n);

        printf("Enter the elements:\n");
        for (i = 0; i < n; i++)
        {
                scanf("%d", &a[i]);
        }

        printf("\nElements at each step:\n");

        for (i = 0; i < n - 1; i++)
        {
                smallest = i;

                for (j = i + 1; j < n; j++)
                {
                        if (a[j] < a[smallest])
                        {
                                smallest = j;
                        }
                }

                temp = a[i];
                a[i] = a[smallest];
                a[smallest] = temp;

                printf("Step %d: ", i + 1);
                for (j = 0; j < n; j++)
                {
                        printf("%d ", a[j]);
                }
                printf("\n");
        }

        printf("\nSorted array: ");
        for (i = 0; i < n; i++)
        {
                printf("%d ", a[i]);
        }

        return 0;
}

```

PROGRAM OUTPUT Execution Verification

```
Enter number of elements: 5\nEnter 5 integers:\n50 20 40 10 30\nSorted list in ascending order:\n10 20 30 40 50
```

PROGRAM 20 `20_quick_sort.c` **Quick Sort**

C SOURCE CODE 20_quick_sort.c

```c
#include <stdio.h>

void display(int arr[], int size)
{
    int i;

    for (i = 0; i < size; i++)
    {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int partition(int arr[], int low, int high, int size)
{
    int pivot = arr[high];
    int i = low - 1;
    int j;
    int temp;

    printf("Pivot is %d\n", pivot);

    for (j = low; j < high; j++)
    {
        if (arr[j] < pivot)
        {
            i++;

            temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;

            printf("After swapping: ");
            display(arr, size);
        }
    }

    temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;

    printf("After placing pivot: ");
    display(arr, size);

    return i + 1;
}

void quickSort(int arr[], int low, int high, int size)
{
    int pivotPosition;

    if (low < high)
    {
        pivotPosition = partition(arr, low, high, size);

        quickSort(arr, low, pivotPosition - 1, size);
        quickSort(arr, pivotPosition + 1, high, size);
    }
}

int main()
{
    int arr[100];
    int size;
    int i;
```

```
66
67      printf("Enter the number of elements: ");
68      scanf("%d", &size);
69
70      printf("Enter the elements:\n");
71      for (i = 0; i < size; i++)
72      {
73          scanf("%d", &arr[i]);
74      }
75
76      printf("\nOriginal array: ");
77      display(arr, size);
78
79      printf("\nSteps of quick sort:\n");
80      quickSort(arr, 0, size - 1, size);
81
82      printf("\nSorted array: ");
83      display(arr, size);
84
85      return 0;
86  }
```

PROGRAM OUTPUT Execution Verification

```
Enter number of elements: 5\nEnter 5 integers:\n50 20 40 10 30\nSorted array:\n10 20 30 40 50
```

PROGRAM 21 `21_merge_sort.c` **Merge Sort**

C SOURCE CODE 21_merge_sort.c

```c
#include <stdio.h>

void printArray(int a[], int n)
{
    int i;

    for (i = 0; i < n; i++)
    {
        printf("%d ", a[i]);
    }
    printf("\n");
}

void merge(int a[], int low, int middle, int high, int n)
{
    int temp[100];
    int i = low;
    int j = middle + 1;
    int k = low;

    while (i ≤ middle && j ≤ high)
    {
        if (a[i] < a[j])
        {
            temp[k] = a[i];
            i++;
        }
        else
        {
            temp[k] = a[j];
            j++;
        }
        k++;
    }

    while (i ≤ middle)
    {
        temp[k] = a[i];
        i++;
        k++;
    }

    while (j ≤ high)
    {
        temp[k] = a[j];
        j++;
        k++;
    }

    for (i = low; i ≤ high; i++)
    {
        a[i] = temp[i];
    }

    printf("After merging positions %d to %d: ", low, high);
    printArray(a, n);
}

void mergeSort(int a[], int low, int high, int n)
{
    int middle;

    if (low < high)
    {
        middle = (low + high) / 2;
```

```c

        mergeSort(a, low, middle, n);
        mergeSort(a, middle + 1, high, n);
        merge(a, low, middle, high, n);
    }
}

int main(void)
{
    int a[100];
    int n;
    int i;

    printf("Enter the number of elements: ");
    scanf("%d", &n);

    if (n ≤ 0 || n > 100)
    {
        printf("Please enter a number between 1 and 100.\n");
        return 1;
    }

    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++)
    {
        scanf("%d", &a[i]);
    }

    printf("\nOriginal array: ");
    printArray(a, n);

    printf("\nSteps:\n");
    mergeSort(a, 0, n - 1, n);

    printf("\nSorted array: ");
    printArray(a, n);

    return 0;
}
```

PROGRAM OUTPUT Execution Verification

```
Enter number of elements: 5\nEnter 5 integers:\n50 20 40 10 30\nSorted array:\n10 20 30 40 50
```